Rish Jain // rish.jain@sjsu.edu // 015731690

Professor Shim

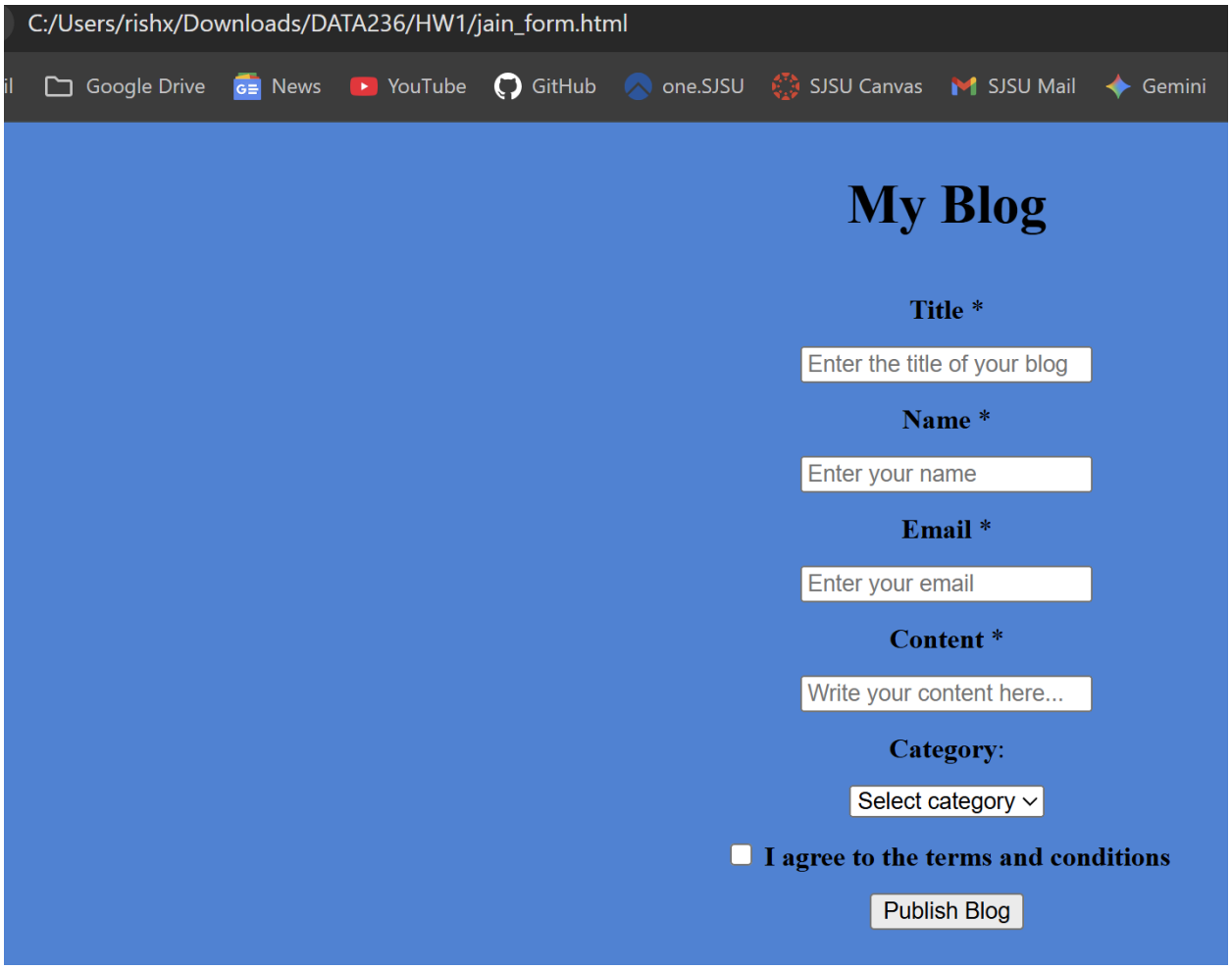
DATA 236

3 February 2026

HW 1

I. HTML

1.



```html
<!DOCTYPE html>

<html>
<head>
    <title>HW 1 - Rish Jain</title>
    <style>
```

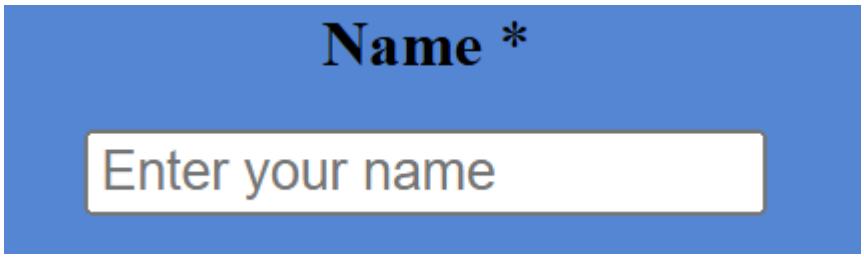
2.


```html
<h1>My Blog</h1>
```

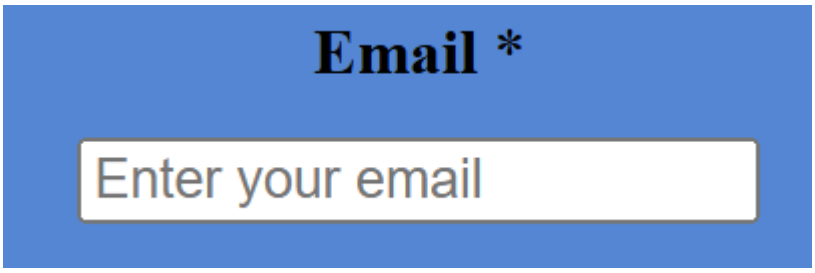
3.

a.

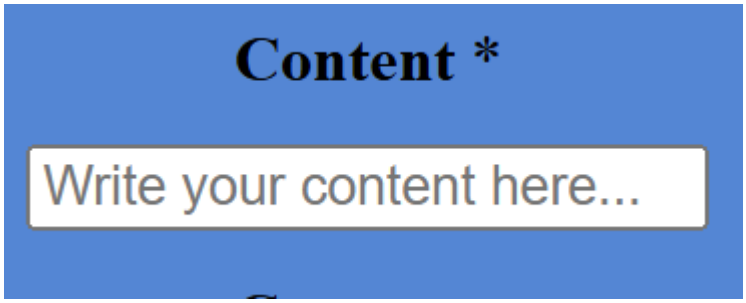
```html
<div><label for="title"><strong>Title</strong> *</label></div>
<div>
    <input
        type="text"
        id="title"
        placeholder="Enter the title of your blog"
        required
        autofocus
    >
</div>
```

b. 

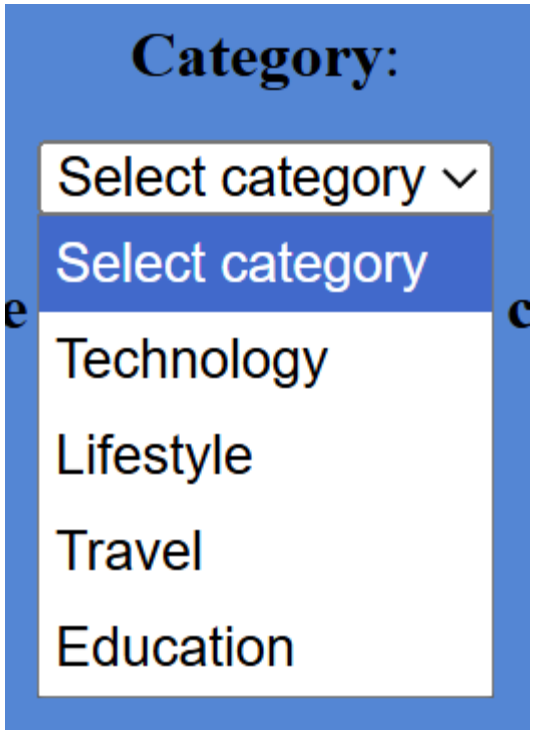

```html
<div><label for="name"><strong>Name</strong> *</label></div>
<div>
    <input
        type="text"
        id="name"
        placeholder="Enter your name"
        required
    >
</div>
```

c. 

```html
<div><label for="email"><strong>Email</strong> *</label></div>
<div>
    <input
        type="text"
        id="email"
        placeholder="Enter your email"
        required
    >
</div>
```

d. 

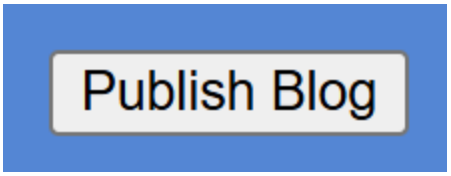

```html
<div><label for="content"><strong>Content</strong> *</label></div>
<div>
    <input
        type="text"
        id="content"
        placeholder="Write your content here..."
        required
    >
</div>
```

e. 


```html
<label for="category"><strong>Category</strong>:</label>
<select id="category" name="category" required>
    <option value="">Select category</option>
    <option value="tech">Technology</option>
    <option value="life">Lifestyle</option>
    <option value="travel">Travel</option>
    <option value="education">Education</option>
</select>
```

f.


```html
<div>
<label><input type="checkbox" id="terms"><strong> I agree to the terms and conditions </strong></label>
</div>
```

g. 


```
<div>
<input
        type="submit"
        value="Publish Blog"
    >
</div>
```

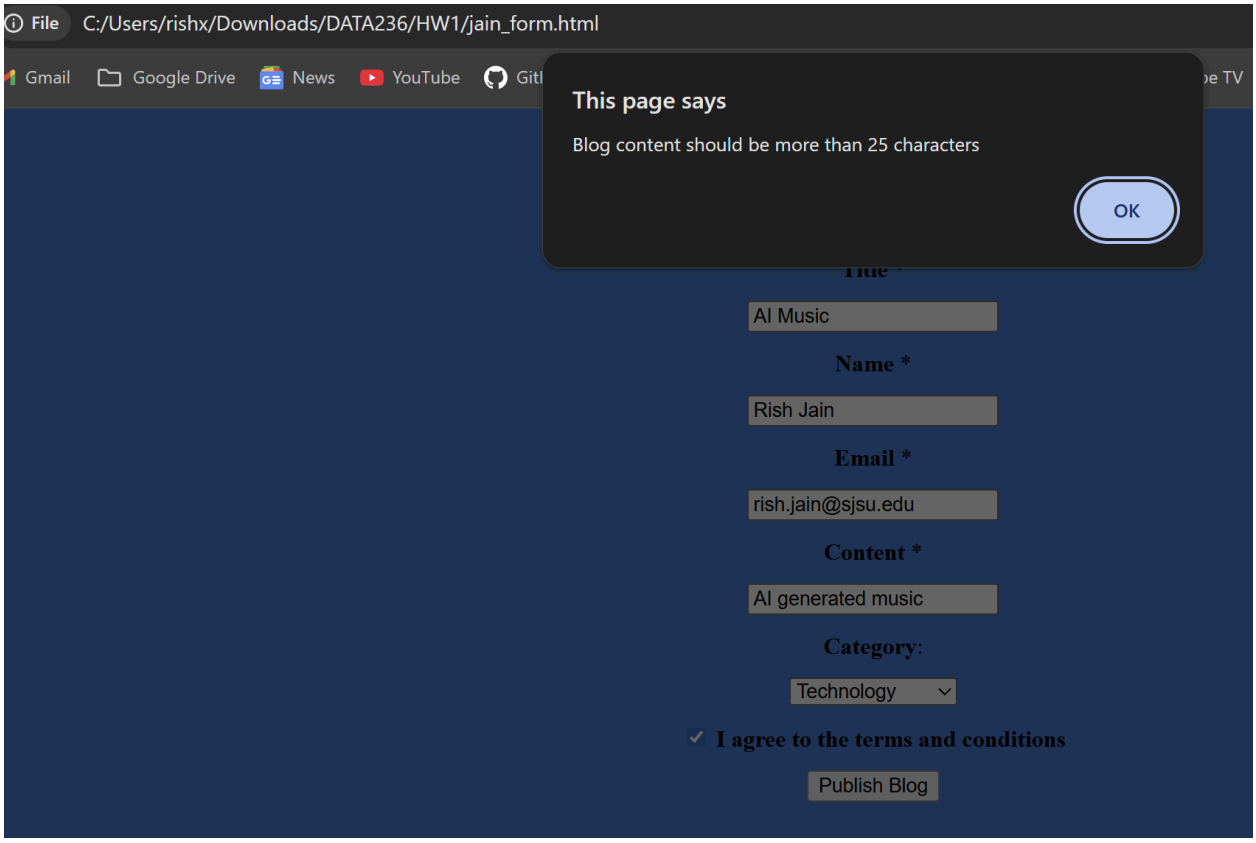
4.

```
    </form>
    <script src="jain_form.js"></script>
</body>
</html>
```

II. Javascript

1.

a.




```
const validateForm = () => {
    const content = document.getElementById("content").value.trim();
    const termsChecked = document.getElementById("terms").checked;

    if (content.length <= 25) {
        alert("Blog content should be more than 25 characters");
        return false;
    }
```

b.

This page says

You must agree to the terms and conditions

OK

Title *

AI Music

Name *

Rish Jain

Email *

rish.jain@sjsu.edu

Content *

AI generated music is a gro

Category:

Technology

☐ **I agree to the terms and conditions**

Publish Blog

```javascript
const validateForm = () => {
    const content = document.getElementById("content").value.trim();
    const termsChecked = document.getElementById("terms").checked;

    if (content.length <= 25) {
        alert("Blog content should be more than 25 characters");
        return false;
    }

    if (!termsChecked) {
        alert("You must agree to the terms and conditions");
        return false;
    }

    return true;
};
```

2.

```
String Blog Data: {                                                    jain_form.js:46
  "title": "AI Music",
  "name": "Rish Jain",
  "email": "rish.jain@sjsu.edu",
  "content": "AI music is a growing concern in today's society",
  "category": "tech",
  "agreedToTerms": true,
  "timestamp": "2026-01-31T21:49:29.934Z"
}
Json Blog Data:                                                        jain_form.js:49
 ▸ {title: 'AI Music', name: 'Rish Jain', email: 'rish.jain@sjsu.edu', content: "AI music
   is a growing concern in today's society", category: 'tech', …}
```

```
document.querySelector("form").addEventListener("submit", (e) => {
    e.preventDefault();

    if (!validateForm()) return;

    const title = document.getElementById("title").value.trim();
    const name = document.getElementById("name").value.trim();
    const email = document.getElementById("email").value.trim();
    const content = document.getElementById("content").value.trim();
    const category = document.getElementById("category").value;
    const termsChecked = document.getElementById("terms").checked;

    const blogData = {
        title,
        name,
        email,
        content,
        category,
        agreedToTerms: termsChecked,
        timestamp: new Date().toISOString()
    };
```

```
const jsonBlogData = JSON.stringify(blogData, null, 2);
console.log("String Blog Data:", jsonBlogData);

const parsedBlogData = JSON.parse(jsonBlogData);
console.log("Json Blog Data:", parsedBlogData);
```

3.

```
Title: AI Music                                    jain_form.js:52
Email: rish.jain@sjsu.edu                          jain_form.js:53
```

```
const { title: parsedTitle, email: parsedEmail } = parsedBlogData;
console.log("Title:", parsedTitle);
console.log("Email:", parsedEmail);
```

4.

```
String Blog Data: {
  "title": "AI Music",
  "name": "Rish Jain",
  "email": "rish.jain@sjsu.edu",
  "content": "AI music is a growing concern in today's society",
  "category": "tech",
  "agreedToTerms": true,
  "timestamp": "2026-01-31T21:49:29.934Z"
}
```

```
const currentCount = blogCounter();
const updatedBlog = {...parsedBlogData, id: `blog-${currentCount}`, submissionDate: new Date().toLocaleString()}

console.log("Blog Counter:", currentCount);
console.log("Updated Blog:", updatedBlog);
```

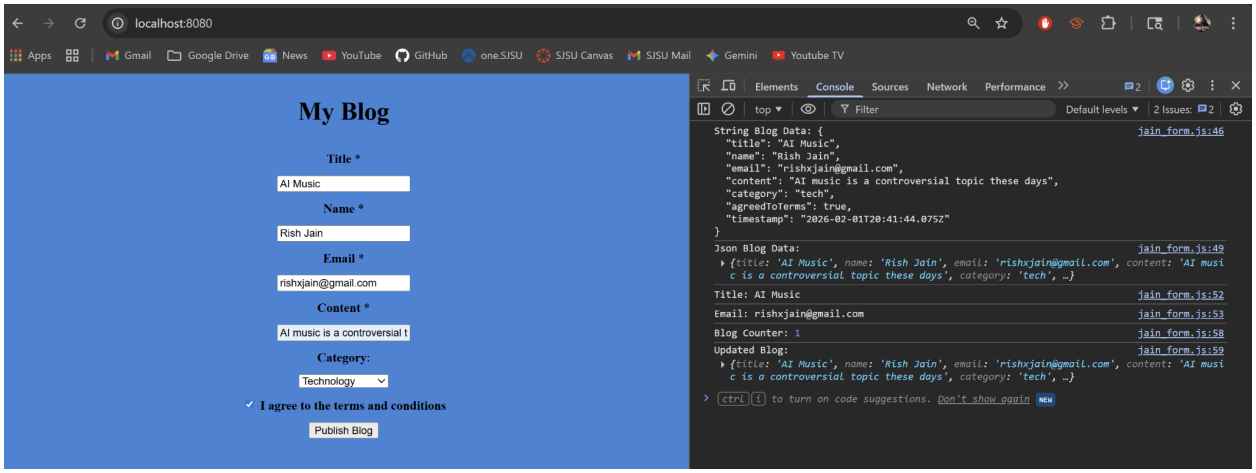
5.

```
String Blog Data: {                                                    jain_form.js:46
  "title": "AI Music",
  "name": "Rish Jain",
  "email": "rish.jain@sjsu.edu",
  "content": "AI music is a growing concern in today's society",
  "category": "tech",
  "agreedToTerms": true,
  "timestamp": "2026-01-31T21:49:29.934Z"
}
Json Blog Data:                                                        jain_form.js:49
 ▸ {title: 'AI Music', name: 'Rish Jain', email: 'rish.jain@sjsu.edu', content: "AI music
   is a growing concern in today's society", category: 'tech', …}
Title: AI Music                                                        jain_form.js:52
Email: rish.jain@sjsu.edu                                              jain_form.js:53
Blog Counter: 1                                                        jain_form.js:58
Updated Blog:                                                          jain_form.js:59
 ▸ {title: 'AI Music', name: 'Rish Jain', email: 'rish.jain@sjsu.edu', content: "AI music
   is a growing concern in today's society", category: 'tech', …}
```

```
const blogCounter = (() => {
    let count = 0;
    return () => ++count;
    })();
```

Deployment:

Docker:




```
PS C:\Users\rishx\Downloads\DATA236\HW1> mkdir my-web-app


    Directory: C:\Users\rishx\Downloads\DATA236\HW1


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----          2/1/2026   12:35 PM                my-web-app


PS C:\Users\rishx\Downloads\DATA236\HW1> cd my-web-app
```

```
PS C:\Users\rishx\Downloads\DATA236\HW1\my-web-app> docker build -t my-web-app .
 => => transferring context: 4.47kB
 => CACHED [2/3] RUN rm -rf /usr/share/nginx/html/*
 => [3/3] COPY . /usr/share/nginx/html/
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:b7417c964aa99b0a522881b9614e634ccdd405aec5e0f815f8448f0c90fb5413
 => => exporting config sha256:0d24fdfea3de720abc07918bc0fa5d7abd3832498917ce87c337b466ff4b6b2f
 => => exporting attestation manifest sha256:bbe3d2eed9f97217122677e82dea97b3a7ff8776aa806d1ec980517ba138dede
 => => exporting manifest list sha256:35b9af293d7e22142747bc6f70e7a73624ee2e82486adbd1ceffb64f07443dbf
 => => naming to docker.io/library/my-web-app:latest
 => => unpacking to docker.io/library/my-web-app:latest
PS C:\Users\rishx\Downloads\DATA236\HW1\my-web-app> docker run -d -p 8080:80 --name my-web-app-container my-web-app
bf2c7ce39078e9135996df1498088d16bdc38263efcf7c217b0269ef8b1bd98a
PS C:\Users\rishx\Downloads\DATA236\HW1\my-web-app> docker stop my-web-app-container
my-web-app-container
PS C:\Users\rishx\Downloads\DATA236\HW1\my-web-app> docker rm my-web-app-container
my-web-app-container
PS C:\Users\rishx\Downloads\DATA236\HW1\my-web-app>
```

```
Dockerfile > ...
# Use Nginx as the base image
FROM nginx:alpine

# Remove default Nginx HTML files
RUN rm -rf /usr/share/nginx/html/*

# Copy all frontend files (HTML, CSS, JS) into the Nginx web root
COPY . /usr/share/nginx/html/

# Expose port 80
EXPOSE 80

# Start Nginx
CMD ["nginx", "-g", "daemon off;"]
```

AWS ECS:

Successfully created private repository, my-web-app

Private repositories (1)

View push commands | Delete | Actions | Create repository

Search by repository substring

Repository name	URI	Created at	Tag immutability	Encryption type
my-web-app	358304212325.dkr.ecr.us-west-1.amazonaws.com/my-web-app	February 01, 2026, 13:08:49 (UTC-08)	Mutable	AES-256

```
PS C:\Users\rishx> aws ecr get-login-password --region us-west-1 | docker login --username AWS --password-stdin 35830421
2325.dkr.ecr.us-west-1.amazonaws.com
Login Succeeded
PS C:\Users\rishx> docker tag my-web-app:latest 358304212325.dkr.ecr.us-west-1.amazonaws.com/my-web-app:latest
PS C:\Users\rishx> docker push 358304212325.dkr.ecr.us-west-1.amazonaws.com/my-web-app:latest
The push refers to repository [358304212325.dkr.ecr.us-west-1.amazonaws.com/my-web-app]
d98afadad42c: Pushed
263f74561a9c: Pushed
6c2f4c63521d: Pushed
211bae00ea56: Pushed
9739627526d7: Pushed
eacf72b1ee7b: Pushed
76fc765fd3b0: Pushed
589002ba0eae: Pushed
cd835559902a: Pushed
9331cd6029cb: Pushed
55cbbd6285fe: Pushed
latest: digest: sha256:35b9af293d7e22142747bc6f70e7a73624ee2e82486adbd1ceffb64f07443dbf size: 856
PS C:\Users\rishx>
```

EC2 > Security Groups > Create security group

Create security group Info

A security group acts as a virtual firewall for your instance to control inbound and outbound traffic. To create a new security group, complete the fields below.

Basic details

Security group name Info

my-web-app-sg

Name cannot be edited after creation.

Description Info

Security group for my web app

VPC Info

vpc-0ba3c6b49ec67d356

Inbound rules Info

Type Info	Protocol Info	Port range Info	Source Info	Description - optional Info	
HTTP	TCP	80	Any... 0.0.0.0/0		Delete

Add rule

Security group (sg-0066b57882df6be5a | my-web-app-sg) was created successfully

Details

sg-0066b57882df6be5a - my-web-app-sg

Actions

Cluster	Services	Tasks	Container instances	CloudWatch monitoring	Cap
my-web-app-cluster	0	No tasks running	0 EC2	Default	No

Task definition successfully created

my-web-app-task:1 has been successfully created. You can use this task definition to deploy a service or run a task.

View task definition

my-web-app-task:1

Last updated February 1, 2026, 13:42 (UTC-8:00)

Deploy | Actions | Create new revision

Overview Info

ARN
arn:aws:ecs:us-west-1:358304212325:task-definition/my-web-app-task:1

Status
ACTIVE

Time created
February 1, 2026, 13:42 (UTC-8:00)

App environment
Fargate

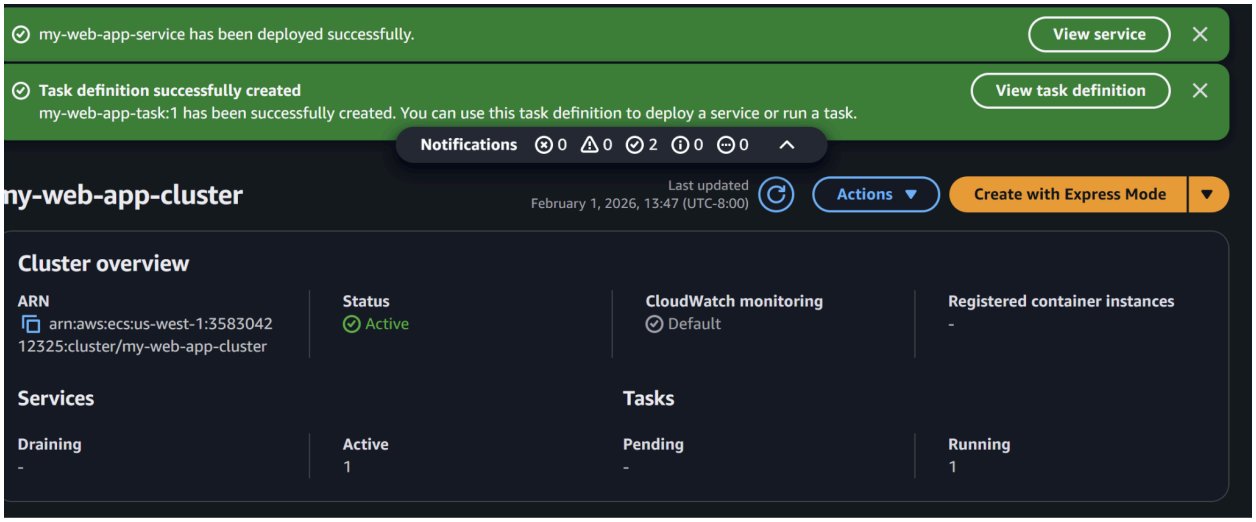
Task role
-

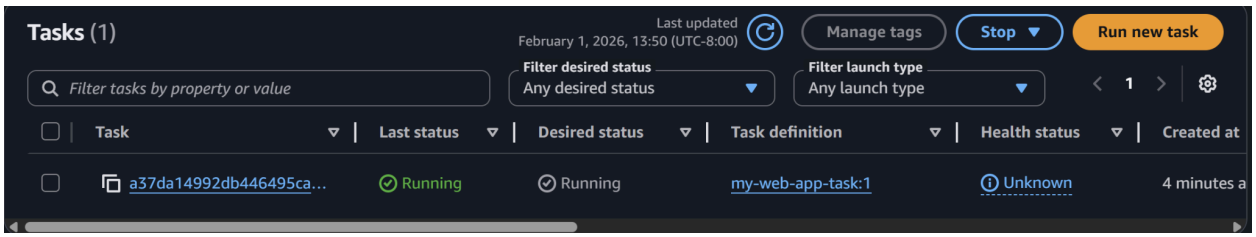
Task execution role
ecsTaskExecutionRole

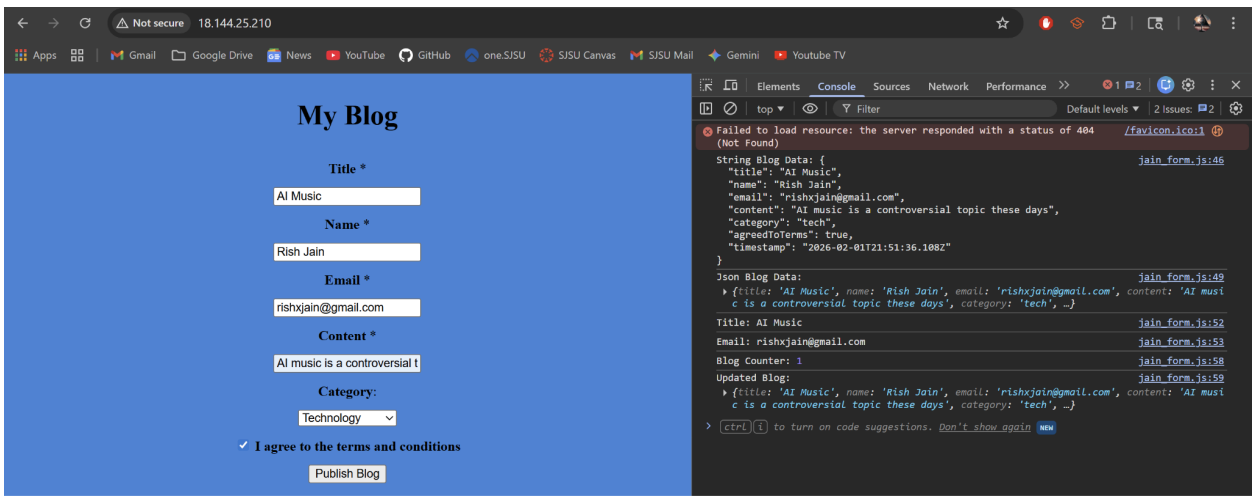
Operating system/Architecture
Linux/X86_64

Network mode
awsvpc

Fault injection
Turned off







Agentic AI

1.

```
def run_agents(title, content):
    planner_agent = [SystemMessage(content=("""You are supposed to generate
        metadata for a blog post. Return ONLY JSON that matches the provided schema.
```

```python
if __name__ == "__main__":
    blog_title = "The Rise of AI Music: When Code Meets Creativity"
    blog_content = ("""AI music is changing the way we create and expe

For artists, AI is less about replacement and more about collaboratior

As AI music continues to evolve, it raises exciting questions about cr
""")
    run_agents(blog_title, blog_content)
```

2.

```
Planner Output: {'tags': ['AI Music', 'Machine Learning', 'Generative Models'], 'summary': 'The rise of AI music is changing
the way we create and experience sound, with machine learning algorithms generating entirely new music in seconds. By using m
achine learning, AI can analyze thousands of songs, understand patterns in melody and rhythm, and generate entirely new music
 in seconds.'}
Reviewer Output: {'tags': ['AI Music', 'Machine Learning', 'Generative Models'], 'summary': 'The rise of AI music is changing
 the way we create and experience sound, with machine learning algorithms generating entirely new music in seconds. What once
 took hours in a studio can now start with a simple prompt.'}
Final Output: {'tags': ['AI Music', 'Machine Learning', 'Generative Models'], 'summary': 'The rise of AI music is changing th
e way we create and experience sound, with machine learning algorithms generating entirely new music in seconds. What once to
ok hours in a studio can now start with a simple prompt.'}
```

```python
def run_agents(title, content):
    planner_agent = [SystemMessage(content=("""You are supposed to generate
        metadata for a blog post. Return ONLY JSON that matches the provided schema.
        NO code. The one-sentence summary must be less than or equal to 25 words and
        it should paraphrase the blog content without copying any phrases verbatim.
        IMPORTANT: The summary must NOT contain the title text, must NOT start with the title,
        and must NOT restate the title.
        Example: If the title is 'How to Bake a Cake',
        the summary should be 'Step-by-step guide to baking delicious cakes at home.'""")),
        HumanMessage(content=f"""Generate 3 topical tags based on the {title}and provide a one-sentence summary
        based on the {content} that has a maximum of 25 words Exclude the title from the summary.""")]

    planner_output = llm.invoke(planner_agent).content
    planner = json.loads(planner_output)
    print("Planner Output:", planner)

    reviewer_agent = [SystemMessage(content=("""Review the planner output to ensure that
        the tags are relevant and the one-sentence summary should be less than or equal to 25 words. The summary
        must NOT contain the title text, must NOT start with the title, and must NOT restate the title. Do NOT
        copy paste the content verbatim in the summary. If any of these conditions are not met, fix them.""")),
        HumanMessage(content=json.dumps({"title": title, "content": content, "draft": planner}))]

    reviewer_output = llm.invoke(reviewer_agent).content
```

```python
def run_agents(title, content):
    reviewer_output = llm.invoke(reviewer_agent).content
    reviewer = json.loads(reviewer_output)
    print("Reviewer Output:", reviewer)

    finalizer_agent = [SystemMessage(content=("""Make sure that the output is strictly JSON and adheres
        to the schema""")),
        HumanMessage(content=json.dumps({"draft": reviewer}))]

    finalizer_output = llm.invoke(finalizer_agent).content
    finalizer = json.loads(finalizer_output)
    print("Final Output:", finalizer)

    print("\nFinal JSON:")
    print(json.dumps(finalizer, indent=2))
```

3.

a. Tags were AI Music, Machine Learning and Generative Models.

b. The rise of AI music is changing the way we create and experience sound, with machine learning algorithms generating entirely new music in seconds. What once took hours in a studio can now start with a simple prompt.

c. The Reviewer agent did change the summary. I told it to make improvements and it did reduce the summary, but could not keep it below 25 words despite all the instructions I gave it. I had to implement a schema to make the outputs JSON because no matter if I said - do not write code, just write text - it would still give code for the tags and the summary. The model can follow basic text instructions like improve the summary but it does not follow those with no code and json outputs without a schema. The main issue, however, was that the model smollm:1.7b was very small that it could not follow proper instructions; using this model it was impossible to follow the instructions to keep the summary below 25 words and have the output be JSON. The model can't come up with its own creative summary or words as a better model should've been chosen for the requirements of this assignment.

Explanation of steps:

```python
import json
from langchain_ollama import ChatOllama
from langchain_core.messages import HumanMessage, SystemMessage

SCHEMA = {
    "type": "object",
    "properties": {
        "tags": {"type": "array", "items": {"type": "string"}, "minItems": 3, "maxItems": 3},
        "summary": {"type": "string"}
    },
    "required": ["tags", "summary"],
    "additionalProperties": False
}

llm = ChatOllama(model="smollm:1.7b", temperature=0, format=SCHEMA)
```

These first import the necessary libraries and then set the schema. The model chosen cannot follow basic instructions to even make json so I had to force-feed it a json schema; I worked with this in my previous internship. The schema ensures that the output is json with 3 tags and a summary, there are no less and no more than 3 tags which is an array of strings (there needs to be 3 in one output), and a summary which is a string. The llm model is set to smollm:1.7b and the output should be the json schema.

```python
def run_agents(title, content):
    planner_agent = [SystemMessage(content=("""You are supposed to generate
        metadata for a blog post. Return ONLY JSON that matches the provided schema.
        NO code. The one-sentence summary must be less than or equal to 25 words and
        it should paraphrase the blog content without copying any phrases verbatim.
        IMPORTANT: The summary must NOT contain the title text, must NOT start with the title,
        and must NOT restate the title.
        Example: If the title is 'How to Bake a Cake',
        the summary should be 'Step-by-step guide to baking delicious cakes at home.'""")),
        HumanMessage(content=f"""Generate 3 topical tags based on the {title}and provide a one-sentence summary
        based on the {content} that has a maximum of 25 words Exclude the title from the summary.""")]

    planner_output = llm.invoke(planner_agent).content
    planner = json.loads(planner_output)
    print("Planner Output:", planner)

    reviewer_agent = [SystemMessage(content=("""Review the planner output to ensure that
        the tags are relevant and the one-sentence summary should be less than or equal to 25 words. The summary
        must NOT contain the title text, must NOT start with the title, and must NOT restate the title. Do NOT
        copy paste the content verbatim in the summary. If any of these conditions are not met, fix them.""")),
        HumanMessage(content=json.dumps({"title": title, "content": content, "draft": planner}))]

    reviewer_output = llm.invoke(reviewer_agent).content
    reviewer = json.loads(reviewer_output)
    print("Reviewer Output:", reviewer)

    finalizer_agent = [SystemMessage(content=("""Make sure that the output is strictly JSON and adheres
        to the schema""")),
        HumanMessage(content=json.dumps({"draft": reviewer}))]

    finalizer_output = llm.invoke(finalizer_agent).content
    finalizer = json.loads(finalizer_output)
    print("Final Output:", finalizer)

    print("\nFinal JSON:")
    print(json.dumps(finalizer, indent=2))
```

This is a function to run all of the agents. I chose 3 agents instead of 2 because the output is already json - the finalizer only outputs the json schema. The function takes in a title and content. Each agent has both a System and Human messages which is like rules vs input. The SystemMessage sets the strict rules to follow while the HumanMessage gives the context. The first agent, the planner tells the model to only output JSON, gives an example of what the summary should look like, and gives rules for the summary; after a lot of testing I ended up adding all of these rules in there for it to work. The HumanMessage for the planner provides the title and the content for the context to give 3 tags and a relevant summary. The planner agent is then ran through the llm command and then printed out in the terminal. The reviewer agent reviews the planner output and makes sure that there are only 3 relevant tags and the summary is

concise + less than 25 words. After a lot of testing, it was difficult to get every output to a maximum of 25 words because the recommended model was impossible to use for the assignment not following basic instructions. The SystemMessage gave the rules again to fix the planner while the HumanMessage provided the context with the title, content, and this time the planner output itself. The reviewer agent is then ran through the llm command and then printed out in the terminal. Finally, the output is already JSON but the assignment required a finalizer to ensure JSON; I came up with an agent for a final check which just utilizes a SystemMessage to give a rule that the final output must be JSON and a HumanMessage which provides the context of the reviewer agent to finalize. The finalizer agent is then ran through the llm command and then printed out in the terminal. Last but not least, the finalizer output is printed out as JSON in the terminal through the json.dumps command.

```
if __name__ == "__main__":
    blog_title = "The Rise of AI Music: When Code Meets Creativity"
    blog_content = ("""AI music is changing the way we create and experience sound. By using machine learning, AI ca

For artists, AI is less about replacement and more about collaboration. Musicians use AI tools to spark ideas, explo

As AI music continues to evolve, it raises exciting questions about creativity and authorship. One thing is clear: t
""")
    run_agents(blog_title, blog_content)
```

This block of main code provides the title and content and calls the function to run with the parameters of the provided title and content.